

Corporate Programs - Code Explanation (Hinglish)

Yeh folder ek "Corporate Sponsored Degree & Training Registration System" implement karta hai. Isme alag-alag companies (jaise TCS, Wipro) apne employees ya candidates ke liye sponsored programs (MCA, M.Tech) run karti hain, jisme eligibility aur selection criteria apply hota hai.

1. AcademicRecord.h

Yeh class candidate ki academic details (degree, CGPA, passing year) store karti hai.

```
#pragma once
#include <string>

using namespace std;

class AcademicRecord {
    string degree;
    double cgpa;
    int graduationYear;

public:
    // Constructor
    AcademicRecord(string degree, double cgpa, int graduationYear)
        : degree(degree), cgpa(cgpa), graduationYear(graduationYear) {}

    // Getters for properties
    double getCgpa() const { return cgpa; }
    string getDegree() const { return degree; }
    int getGraduationYear() const { return graduationYear; }
};
```

2. Candidate.h

Candidate class ek applicant ki profile maintain karti hai (naam, email, aur uska AcademicRecord).

```
#pragma once
#include <string>
#include "AcademicRecord.h"

using namespace std;

class Candidate {
    string name;
    string email;
    AcademicRecord record;

public:
```

```
// Constructor initialize karta hai candidate details
Candidate(string name, string email, AcademicRecord record)
    : name(name), email(email), record(record) {}

string getName() const { return name; }
const AcademicRecord& getRecord() const { return record; }
string getEmail() const { return email; }
};
```

3. CorporateProgram.h

Yeh class corporate program ki details define karti hai, jaise program ka naam, category, aur duration (mahine mein).

```
#pragma once
#include <string>

using namespace std;

class CorporateProgram {
    string programName;
    string category;
    int durationMonths;

public:
    // Constructor
    CorporateProgram(string programName, string category, int
durationMonths)
        : programName(programName), category(category),
durationMonths(durationMonths) {}

    string getProgramName() const { return programName; }
    string getCategory() const { return category; }
    int getDurationMonths() const { return durationMonths; }
};
```

4. Company.h

Company class ek specific company ko represent karti hai (e.g. TCS) aur list maintain karti hai un corporate programs ki jo woh company offer karti hai.

```
#pragma once
#include <string>
#include <vector>
#include <memory>
#include "CorporateProgram.h"

using namespace std;
```

```

class Company {
    string name;
    // Pointers ka use kiya gaya hai programs ko memory leak se bachane ke
    // liye (smart pointers)
    vector<shared_ptr<CorporateProgram>> offeredPrograms;

public:
    Company(string name) : name(name) {}

    // Ek naya program add karne ka function
    void addProgram(shared_ptr<CorporateProgram> program) {
        offeredPrograms.push_back(program);
    }

    string getName() const { return name; }
};

```

5. SelectionCriteria.h

Yeh ek abstract base class (interface) hai jo selection ke rules ko define karne ka structure deti hai. Isko inherit karke hum apne hisab se rules bana sakte hain.

```

#pragma once
#include "Candidate.h"

class SelectionCriteria {
public:
    // Pure virtual functions jo sub-classes me implement honge
    virtual bool isEligible(const Candidate& candidate) const = 0;
    virtual double calculateScore(const Candidate& candidate) const = 0;
    virtual ~SelectionCriteria() = default;
};

```

6. StandardSelectionCriteria.h

Yeh class `SelectionCriteria` ko implement karti hai. Isme rule banaya gaya hai ki candidate ka minimum CGPA kitna hona chahiye aur uski degree allowed hai ya nahi.

```

#pragma once
#include <vector>
#include <string>
#include <algorithm>
#include "SelectionCriteria.h"

using namespace std;

class StandardSelectionCriteria : public SelectionCriteria {

```

```

    double minCgpa;
    vector<string> allowedDegrees;

public:
    StandardSelectionCriteria(double minCgpa, const vector<string>&
allowedDegrees)
        : minCgpa(minCgpa), allowedDegrees(allowedDegrees) {}

    // Check karta hai ki degree allowed list mein hai ya nahi aur CGPA
    minimum limit meet karta hai ya nahi
    bool isEligible(const Candidate& candidate) const override {
        const AcademicRecord& rec = candidate.getRecord();
        bool degreeAllowed = find(allowedDegrees.begin(),
allowedDegrees.end(), rec.getDegree()) != allowedDegrees.end();
        return rec.getCgpa() >= minCgpa && degreeAllowed;
    }

    // Sirf CGPA return karta hai as a score
    double calculateScore(const Candidate& candidate) const override {
        return candidate.getRecord().getCgpa();
    }
};

```

7. RecruitmentProcess.h

Yeh sabse important logic wali class hai jo poora recruitment cycle manage karti hai: applications receive karna, eligibility check karna, aur merit (CGPA) ke basis pe final selection list banana.

```

#pragma once
#include <iostream>
#include <vector>
#include <memory>
#include <algorithm>
#include "Company.h"
#include "CorporateProgram.h"
#include "SelectionCriteria.h"
#include "Candidate.h"

using namespace std;

class RecruitmentProcess {
    shared_ptr<Company> company;
    shared_ptr<CorporateProgram> program;
    int batchYear;
    int intakeCapacity; // Kitne bachche lene hain maximum
    shared_ptr<SelectionCriteria> criteria;
    vector<Candidate*> applicants;
    vector<Candidate*> selectedCandidates;

public:
    RecruitmentProcess(shared_ptr<Company> company,

```

```

shared_ptr<CorporateProgram> program, int batchYear, int intakeCapacity,
shared_ptr<SelectionCriteria> criteria)
    : company(company), program(program), batchYear(batchYear),
intakeCapacity(intakeCapacity), criteria(criteria) {}

// Naya candidate apply karega
void apply(Candidate& candidate) {
    applicants.push_back(&candidate);
}

// Eligibility verify karke select karna based on scores
void processSelection() {
    vector<Candidate*> eligible;
    for (Candidate* c : applicants) {
        if (criteria->isEligible(*c)) {
            eligible.push_back(c);
        }
    }

    // Descending order me sort karna (High CGPA wala pehle)
    sort(eligible.begin(), eligible.end(), [this](Candidate* c1,
Candidate* c2) {
        return criteria->calculateScore(*c1) > criteria-
>calculateScore(*c2);
    });

    cout << "--- Selection List for " << company->getName() << " " <<
program->getProgramName() << " (" << batchYear << ") ---" << endl;
    int count = 0;

    // Intake capacity ke hisab se select karna
    for (Candidate* c : eligible) {
        if (count < intakeCapacity) {
            selectedCandidates.push_back(c);
            cout << c->getName() << " - Selected with CGPA: " << c-
>getRecord().getCgpa() << endl;
            count++;
        }
    }
    if (selectedCandidates.empty()) {
        cout << "No candidates met the eligibility criteria or
applied." << endl;
    }
    cout << "-----" <<
endl;
}
};

```

8. main.cpp

Yeh file entry point hai jo in sabhi objects ko instantiate karti hai. Do companies (TCS aur Wipro) ke programs banati hai, students se apply karwati hai aur selection process chalati hai.

```
#include <iostream>
#include <memory>
#include <vector>
#include <string>
#include "AcademicRecord.h"
#include "Candidate.h"
#include "SelectionCriteria.h"
#include "StandardSelectionCriteria.h"
#include "CorporateProgram.h"
#include "Company.h"
#include "RecruitmentProcess.h"

using namespace std;

int main() {
    cout << "=== Corporate Sponsored Degree & Training Registration System\n" << endl;

    // Companies create karna
    auto tcs = make_shared<Company>("TCS");
    auto wipro = make_shared<Company>("Wipro");

    // Corporate programs create karna
    auto tcsIgnite = make_shared<CorporateProgram>("Ignite MCA", "MCA",
24);
    auto wiproWilp = make_shared<CorporateProgram>("WILP M.Tech", "MTECH",
48);

    tcs->addProgram(tcsIgnite);
    wipro->addProgram(wiproWilp);

    // Candidates aur unke academic records banana
    AcademicRecord rec1("BCA", 8.5, 2024);
    Candidate c1("Ramesh", "ramesh@email.com", rec1);

    AcademicRecord rec2("BSC", 9.2, 2024);
    Candidate c2("Suresh", "suresh@email.com", rec2);

    AcademicRecord rec3("BTECH", 7.5, 2024);
    Candidate c3("Anita", "anita@email.com", rec3);

    // Eligibility Criteria set karna alag alag programs ke liye
    auto wilpCriteria = make_shared<StandardSelectionCriteria>(6.0,
vector<string>{"BCA", "BSC"});
    auto igniteCriteria = make_shared<StandardSelectionCriteria>(7.0,
vector<string>{"BCA", "BSC"});

    // Recruitment processes initialize karna (batch year, seats, criteria
pass kar rahe)
    RecruitmentProcess wiproProcess(wipro, wiproWilp, 2024, 1,
wilpCriteria);
    RecruitmentProcess tcsProcess(tcs, tcsIgnite, 2024, 2, igniteCriteria);
```

```
// Candidates ka apply karna
wiproProcess.apply(c1);
wiproProcess.apply(c2);
wiproProcess.apply(c3);

tcsProcess.apply(c1);
tcsProcess.apply(c2);

// Final Selection run karna
wiproProcess.processSelection();
cout << endl;
tcsProcess.processSelection();

return 0;
}
```